

# Using Hypervectors for Efficient Anomaly Detection in Graph Streams

William Arliss  
Leidos  
Arlington, USA  
william.f.arliss@leidos.com

Andrew B Godbehere  
Leidos  
Arlington, USA  
andrew.godbehere@leidos.com

Graham Mueller  
Leidos  
Arlington, USA  
william.g.mueller@leidos.com

**Abstract**—We present an online algorithm for detecting changes in computer network activity. Anomalous activity in IT systems often appear as changes in network topology as edges evolve in a dynamic graph; identifying these behavioral changes can be a challenging task. We propose a method that uses principles of Hyperdimensional Computing to encode graphs to a real valued space where anomalies are easily identifiable. With reasonable assumptions of the baseline edge generating process, our approach operates in real time and can produce an anomaly score primitive for one sample independently of all others. This score lends itself easily to an online Bayesian confidence estimate in constant memory, which is essential for real-world applications where networks are extremely large and interpretable predictions are needed in real time. We demonstrate the effectiveness of our approach on both synthetic and real-world datasets.

**Index Terms**—Anomaly detection, graph sketching, streaming data, hyperdimensional computing

## I. INTRODUCTION

The problem of identifying windows of abnormal activity in relational data is pertinent to many domains and is particularly relevant to IT monitoring systems. With the increased adoption of Zero Trust [1] frameworks for network access control, the need for reliable real-time network intrusion and anomaly detection is becoming increasingly important.

Identifying network anomalies among a majority of benign traffic is a task well suited to graph theory. A computer network is naturally represented as a dynamic graph where nodes correspond to endpoints and edges correspond to communications between endpoints which can evolve over time. Network anomalies manifest as the appearance of edges that deviate from the baseline topology of the graph (see figure 1).

Two related anomaly detection problems for dynamic graphs involve edge streams (a time-ordered sequence of individual edges) and graph streams (a time-ordered sequence of graphs or subgraphs). Very large real-world datasets make near real-time analysis difficult in both of these settings and limit the applicability of deep learning models. Therefore, efficient approximations capable of online updating in sub-linear or bounded memory (i.e., sketches) are desirable. Here, a sketch refers to a low-rank summary, approximation, or compression of a graph statistic.

Vector Symbolic Architectures [2], also known as Hyperdimensional Computing (HDC), have shown promise in online-learning situations due to the “embarrassing parallel” nature

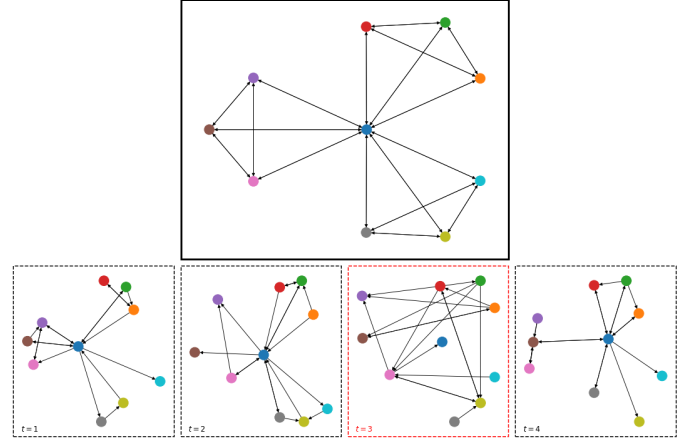


Fig. 1. Sequence of graph snapshots sampled from an underlying windmill graph (top). The snapshot at  $t=3$  is sampled from a complete graph and is therefore considered anomalous.

of the learning operations they use. Recently, HDC has been applied to graph learning tasks. In this work we explore a novel HDC framework for real-time anomaly detection in graph streams as part of a larger Zero Trust system.

### A. Contributions

We propose a method for encoding graphs using HDC operations as well as an anomaly detection technique based on non-parametric hypothesis testing. Our approach is constant in memory, operates in real time, and produces interpretable confidence scores. Furthermore, the encodings we generate are capable of distinguishing key network topologies. Evaluated on several real-world and synthetic datasets, our approach demonstrates improved speed and accuracy against comparison approaches.

## II. RELATED WORK

Graph anomaly detection is surveyed in Ranshous et al. [3] and Akkoglu et al. [4]. A survey of deep learning approaches is offered in [5]. Sketch-based analysis of graph streams receives thorough treatment in [6]–[8]. The present discussion is focused only on sketch-based anomaly detection in edge and graph streams.

### A. Edge Streams

In [9]–[11], variations of the count-min sketch [12] data structure are used to maintain approximations of graph statistics and metrics. The authors use these sketches to identify anomalous activity at the graph, sub-graph, and edge levels. In [13], an efficient algorithm for dense subgraph discovery is proposed in order to detect anomalies in a stream of network edges. Reservoir sampling is used in [14] to monitor changes in the arrival rate of certain edges. Edge frequencies are factorized and stored with constant-memory in [15] to detect anomalies in streaming edges.

### B. Graph Streams

In [16], the frequencies of  $k$ -hop graph traversals are collected in sketch-vectors and locality-sensitive hashing [17] is used to identify outlying graphs in a stream. A higher-order sketch is used in [11] to detect the appearance of dense subgraphs in a dynamic graph. In [18], spectral decomposition of the Laplacian matrix is used to monitor structural change points and anomalous events. Although not sketch-based, [19] also uses spectral decomposition to identify temporally anomalous graphs and nodes in a time series of graphs. In [20], Eswaran et al. use hashing functions to map batches of edges to vector encodings on which an anomaly detection procedure (the authors use [21]) may be applied.

### C. HDC

Popularized by Kanerva [2], HDC is a paradigm of artificial intelligence that represents units of information as very large (hyperdimensional) vectors. HDC is applicable to a wide variety of problem settings [22] through adaptation of the “bind”, “bundle”, and “permute” operations. HDC has recently been introduced to the graph setting by [23] and [24], offering significant speedup and memory reduction on standard graph learning tasks.

Leveraging HDC on graphs, we extend and improve upon the approach of Eswaran et al. [20] by joining the graph encoding and anomaly detection procedures.

## III. METHODS

This section details our approach for encoding graph snapshots, computing anomaly score primitives, and estimating anomaly confidence scores in real time. An anomalous snapshot is one that does not conform to the structure of the underlying graph from which the majority of snapshots are sampled. More specifically, a snapshot containing a significant number of edges that would be unlikely in the underlying graph is an anomaly. This is illustrated in figure 1.

We refer to the full detection procedure as HDSGA (Hyper-Dimensional Streaming Graph Anomaly detection). Complete proofs are provided in appendix A.

### A. Preliminaries

Consider a dynamic graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$  and let  $\{\mathcal{G}_t\}$  be a time-ordered sequence of graph “snapshots” [18] such that

$\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$  is a count-weighted, directed, simple graph composed of the multiset of edges  $\mathcal{E}_t^*$  that traversed  $\mathcal{G}$  in the time window from  $t-1$  to  $t$ . Each node  $u \in \mathcal{V}$  has a unique identifier that is fixed across time windows. Let  $\mathcal{D} = \mathcal{V} \times \mathcal{V}$  be the set of all possible node “dyads”. Assume that every node dyad  $(u, v) \in \mathcal{D}$  has an associated sampling rate  $\alpha_{uv}$  (e.g., the probability of a network endpoint sending or receiving a packet in a given time period) and that the edges in window  $t$  are sampled according to those probabilities. The node set  $\mathcal{V}$  is not assumed to be known in full a priori.

For a given edge  $(u, v, w) \in \mathcal{E}_t$ , let  $u$  be the source node,  $v$  be the destination node, and  $w$  be a random variable corresponding to the number of times the dyad  $(u, v)$  was traversed during the window  $t$ . Expressed as a function of its corresponding dyad,  $w(u, v)_t$  is the non-negative multiplicity of  $(u, v)$  in the multiset  $\mathcal{E}_t^*$ . For convenience, we define the set of edge weights in  $\mathcal{G}_t$  as  $\mathcal{W}_t = \{w : (u, v, w) \in \mathcal{E}_t\}$ . Note that  $|\mathcal{E}_t| = |\mathcal{W}_t| \leq |\mathcal{E}_t^*| = \sum_{w \in \mathcal{W}_t} w$ . Furthermore, we assume  $w(u, v)_t \sim \text{Bin}(|\mathcal{D}|, \alpha_{uv})$  for any  $(u, v) \in \mathcal{E}_t^*$ .

Because  $\mathcal{E}_t^*$  may contain duplicate elements, we consider  $\mathcal{E}_t^*$  to be a sample taken *with* replacement from  $\mathcal{D}$  during window  $t$ . And because  $\mathcal{E}_t$  only contains unique elements,  $\mathcal{E}_t$  is considered a sample *without* replacement where the duplicates are accounted for by  $\mathcal{W}_t$ .

### B. Encoding

The HDC extension of the graph encoding from [20] can be concisely written by first independently assigning each node  $u \in \mathcal{V}$  two vectors  $\hat{\mathbf{H}}_u$  and  $\check{\mathbf{H}}_u$  that remain fixed over time. Let  $\hat{\mathbf{H}}_u \in \{0, 1\}^d$  represent  $u$  as a source in a given dyad and  $\check{\mathbf{H}}_u \in \{0, 1\}^d$  represent it as a destination. For HDC, dimensionality  $d$  is on the order of thousands<sup>1</sup>.

Elements in  $\hat{\mathbf{H}}_u$  and  $\check{\mathbf{H}}_u$  take the value 1 with probabilities  $p$  and  $q$  respectively. Furthermore, these vectors are approximately mutually uncorrelated — that is,  $E[r_{uv}] = 0$ ,  $\forall (u, v) \in \mathcal{D}$ , where  $r_{uv}$  is the empirical Pearson correlation coefficient between  $\hat{\mathbf{H}}_u$  and  $\check{\mathbf{H}}_v$ .

Next, let  $\bar{\mathbf{H}}_{uv} = \hat{\mathbf{H}}_u \odot \check{\mathbf{H}}_v$  be the hypervector representing the  $(u, v)$  dyad, where  $\odot$  is the Hadamard product. With this in mind, we define

$$\begin{aligned} \mathbf{x}_t &= \sum_{(u, v) \in \mathcal{D}} w(u, v)_t \cdot \hat{\mathbf{H}}_u \odot \check{\mathbf{H}}_v \\ &= \sum_{(u, v, w) \in \mathcal{E}_t} w \cdot \bar{\mathbf{H}}_{uv} \end{aligned} \quad (1)$$

as the encoding of  $\mathcal{G}_t$ . We now consider a baseline scenario against which to compare graph snapshots.

**Definition 1.** Let  $\mathcal{G}^\circ = (\mathcal{V}^\circ, \mathcal{E}^\circ)$  be a count-weighted, directed, simple graph sampled from  $\mathcal{G}$  such that each dyad has uniform sampling probability;  $\alpha_{uv} = \alpha$ ,  $\forall (u, v) \in \mathcal{D}$ . Thus  $\mathcal{G}^\circ$  is a random graph with the set of edge weights  $\mathcal{W}^\circ = \{w : (u, v, w) \in \mathcal{E}^\circ\}$ . The encoding of this graph is given by  $\mathbf{x}^\circ$ .

<sup>1</sup>In [20], the authors do not make use of high-dimensionality and instead use  $d = 50$  in their examples.

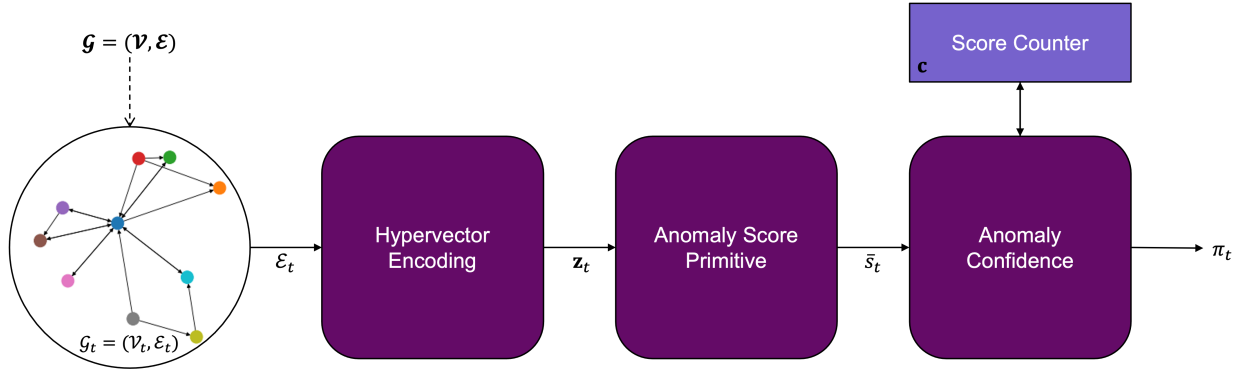


Fig. 2. Pipeline to produce anomaly scores and confidences. A graph snapshot  $\mathcal{G}_t$  is observed and encoded as  $\mathbf{z}_t$  (section III-B). The encoding is used to produce the anomaly primitive score  $\bar{s}_t$  (section III-C). The anomaly confidence  $\pi_t$  is produced by referencing  $\bar{s}_t$  against the counts of historical scores stored in  $\mathbf{c}$  (section III-D).

To examine distributional properties of graph encodings, we focus on the  $i^{\text{th}}$  element of the encoding vector. When referring to the general encoding in the base case from definition 1, this element is denoted  $\mathbf{x}_{\cdot,i}^\circ$ . When referring to a specific encoding at time  $t$ , it is  $\mathbf{x}_{t,i}$ . Note that each element in the encoding vector is distributed identically, so properties of the  $i^{\text{th}}$  element hold for all others.

**Proposition 1.** *If  $\mathcal{G}$  is an infinite graph, then the conditional expectation and variance of the encoding of snapshot  $\mathcal{G}^\circ$  are*

$$\mathbb{E}[\mathbf{x}_{\cdot,i}^\circ | \mathcal{W}^\circ] = pq \sum_{w \in \mathcal{W}^\circ} w \quad (2)$$

$$\text{var}[\mathbf{x}_{\cdot,i}^\circ | \mathcal{W}^\circ] = pq(1-pq) \sum_{w \in \mathcal{W}^\circ} w^2 \quad (3)$$

*Proof.* For any  $(u, v) \in \mathcal{D}$ , note that  $\hat{\mathbf{H}}_{u,i} \sim \text{Ber}(p)$  and  $\hat{\mathbf{H}}_{v,i} \sim \text{Ber}(q)$  and that  $\hat{\mathbf{H}}_u, \hat{\mathbf{H}}_v$  are mutually uncorrelated. This implies  $\hat{\mathbf{H}}_{uv,i} \sim \text{Ber}(pq)$ . Therefore, the conditional distribution of the encoding with respect to the edge weights is

$$\mathbf{x}_{\cdot,i}^\circ | \mathcal{W}^\circ \sim \sum_{w \in \mathcal{W}^\circ} w \cdot \text{Ber}(pq)$$

which is a sum of weighted, independent Bernoulli variables. For the infinite graph, elements in the snapshot of edges (a sample taken without replacement) are exchangeable and independent. That is, the probability of any dyad appearing in  $\mathcal{E}^\circ$  is not affected by the appearance of any other edge. Therefore, the expectation and variance are simply the weighted sums of the expectations and variances of each i.i.d. hypervector.  $\square$

**Proposition 2.** *If  $\mathcal{G}$  is a finite graph, then the conditional expectation of the encoding of snapshot  $\mathcal{G}^\circ$  is given by equation 2 and the conditional variance is*

$$\widetilde{\text{var}}[\mathbf{x}_{\cdot,i}^\circ | \mathcal{W}^\circ] = pq(1-pq) \frac{\lambda}{M-1} \quad (4)$$

$$\lambda = M \sum_{w \in \mathcal{W}^\circ} w^2 - \sum_{w \in \mathcal{W}^\circ} \sum_{g \in \mathcal{W}^\circ} w g \quad (5)$$

where  $M = |\mathcal{D}|$ .

*Proof.* For the finite graph, elements in the snapshot of edges are exchangeable but not independent. That is, the probability that any dyad appears in the edge set is affected by the appearance of other edges before it. Therefore, covariance among elements must be taken into account. A well-known result of finite sampling theory shows that the covariance between two identical Bernoulli variables sampled without replacement from a population of size  $M$  is  $\frac{-\sigma^2}{M-1}$ . In our case,  $\sigma^2 = pq(1-pq)$ . The encoding is therefore a sum of weighted, non-independent Bernoulli variables. Thus, equation 4 is the weighted sum of the variances and covariances of each (non-independent) identically distributed hypervector.  $\square$

**Proposition 3.** *For the infinite random graph  $\mathcal{G}^\circ$ , each element in the standardized encoding approaches the standard normal distribution:*

$$\mathbf{z}_{\cdot,i}^\circ = \frac{\mathbf{x}_{\cdot,i}^\circ - \mathbb{E}[\mathbf{x}_{\cdot,i}^\circ | \mathcal{W}^\circ]}{\sqrt{\text{var}[\mathbf{x}_{\cdot,i}^\circ | \mathcal{W}^\circ]}} \rightarrow \mathcal{N}(0, 1) \quad (6)$$

*Proof.* Using the unconditional expectation and variance, the standardized encoding  $(\mathbf{x}_{\cdot,i}^\circ - \mathbb{E}[\mathbf{x}_{\cdot,i}^\circ]) / \text{var}[\mathbf{x}_{\cdot,i}^\circ]^{\frac{1}{2}}$  approaches  $\mathcal{N}(0, 1)$  by the Lyapunov Central Limit Theorem (assuming weights as givens). The form of this standardization is equivalent to the form of the conditional standardization  $\mathbf{z}_{\cdot,i}^\circ$  given the weight set. Asymptotic normality across the full (unconditional) distribution of  $\mathbf{x}_{\cdot,i}^\circ$  implies normality in the region of the (conditional) distribution given by  $\mathbf{x}_{\cdot,i}^\circ | \mathcal{W}^\circ$  as long as the weights are bounded. Therefore, the conditionally standardized encoding  $\mathbf{z}_{\cdot,i}^\circ$  also approaches  $\mathcal{N}(0, 1)$ .  $\square$

Considering these properties, let

$$\begin{aligned} \mathbf{z}_t &= \frac{\mathbf{x}_t - \mathbb{E}[\mathbf{x}_t | \mathcal{W}_t]}{\sqrt{\text{var}[\mathbf{x}_t | \mathcal{W}_t]}} \\ &= \frac{\mathbf{x}_t - pq \sum_{w_t} w}{\sqrt{pq(1-pq) \sum_{w_t} w^2}} \end{aligned} \quad (7)$$

be the standardized encoding of the graph at time  $t$ . Here  $\text{var}[\cdot]$  may be substituted for  $\widetilde{\text{var}}[\cdot]$  if called for by assumption.

For an arbitrary graph  $\mathcal{G}_t$ , the elements of  $\mathbf{z}_t$  can be used to characterize the generating process of its edges. That is, the standardized encoding of a graph with nearly uniform sampling probabilities will be distributed more closely to standard normal than a graph with highly non-uniform sampling probabilities.

The encoding  $\mathbf{z}_t$  is considered a sketch of snapshot  $\mathcal{G}_t$ , as it is computed in real time and summarizes structural information of the graph. The construct of multiple hash functions accumulated in a vector is common across many sketching algorithms. Streaming computation is possible due to the low overhead of the algorithm, which runs in  $O(dk)$  time. A vector of dimension  $d$  is created for a total of  $k$  edges. Subsequent computations operate on each dimension independently. In practice, computation time can be reduced further by updating the sketch only for edges that change in a given observation.

### C. Anomaly Detection

While the baseline generating process of the snapshots in  $\{\mathcal{G}_t\}$  may not be known, the root mean squared error (RMSE) of the snapshots with respect to the completely random generating process (definition 1) can be measured using  $\mathbf{z}_t$ . Let

$$\begin{aligned} s_t &= \sqrt{\frac{1}{d} \sum_{i=1}^d (\mathbf{z}_{t,i} - \mathbb{E}[\mathbf{z}_{\cdot,i}^\circ])^2} \\ &= \sqrt{\frac{1}{d} \sum_{i=1}^d \mathbf{z}_{t,i}^2} \end{aligned} \quad (8)$$

be the RMSE of  $\mathbf{z}_t$  where  $d$  is the dimension of the encoding. If  $\mathcal{G}_t$  conforms to the generating process of  $\mathcal{G}^\circ$ , then  $s_t$  will be close to 1.

Comparing the set of RMSEs corresponding to  $\{\mathcal{G}_t\}$  allows one to project the anomaly detection problem from the topological space to the much simpler  $\mathbb{R}^1$  space. There are many single-variable anomaly detection techniques that may be applied once  $\{s_t\}$  has been computed. See [25] or [26] for comprehensive surveys. The technique we propose makes use of the high dimensionality of  $\{\mathbf{z}_t\}$  to estimate anomaly confidence scores in a streaming setting.

Our approach initially relies on the assumption that anomalous graphs are relatively *less* random than the baseline. With such an assumption, we may adopt a method similar to non-parametric hypothesis testing with the null hypothesis that a given snapshot is sampled with constant probabilities from an underlying graph. The null can be tested by simply counting the number of elements in  $\mathbf{x}_t$  that are  $\theta$  standard deviations from expectation. This value is found by defining a binarized version of  $\mathbf{z}_t$  as

$$\mathbf{b}_{t,i} = \mathbb{1}_{\{|\mathbf{z}_{t,i}| > \theta\}} \quad (9)$$

where  $\mathbb{1}_{\{\cdot\}}$  is the indicator function.

Summing the elements of  $\mathbf{b}_t$  gives the number of times the null hypothesis is rejected in a two-tailed test with critical

---

### Algorithm 1 Streaming anomaly scores and confidences

---

**Input:**  $\{\mathcal{G}_t\}_T, p, q, \theta_L, \theta_U, \gamma$

**Output:**  $\{\bar{s}_t, \pi_t\}$

```

1:  $\mathbf{c} \leftarrow \mathbf{0}^{d+1}$ 
2: for  $t = 1, \dots, T$  do
3:   Receive the current graph snapshot  $\mathcal{G}_t$ 
4:    $\bar{s}_t \leftarrow \text{AnomalyScore}(\mathcal{G}_t; p, q, \theta_L, \theta_U)$  // Equation 10
5:    $\rho_t \leftarrow (1 + \sum_{i=1}^{\bar{s}_t+1} \mathbf{c}_i) / (2 + (t-1))$ 
6:    $\pi_t \leftarrow 1 - F_{\text{Bin}}[(t-1)(1-\gamma); (t-1), \rho_t]$ 
7:   return  $\bar{s}_t, \pi_t$  // Yield current score/confidence
8:    $\mathbf{c}_{\bar{s}_t+1} \leftarrow \mathbf{c}_{\bar{s}_t+1} + 1$ 
9: end for
```

---

value  $\theta$  repeated  $d$  times. More rejections of the null suggests stronger evidence against  $\mathcal{G}_t$  being a random graph, and therefore stronger evidence that it is an anomaly.

If we instead assume that that anomalous graphs are relatively *more* random than the baseline, an equivalence test is appropriate. We use the “two one-sided tests” procedure [27] to address this alternate assumption; this is achieved by simply reversing the inequality in equation 9.

In order to capture anomalies under both of these assumptions, we define

$$\bar{s}_t = \max \left( \sum_{i=1}^d \mathbb{1}_{\{|\mathbf{z}_{t,i}| < \theta_L\}}, \sum_{i=1}^d \mathbb{1}_{\{|\mathbf{z}_{t,i}| > \theta_U\}} \right) \quad (10)$$

where  $\theta_L \leq \theta_U$ . This value is referred to as the anomaly score primitive and is computed independently of all other graphs observed; it is therefore memoryless.

### D. Confidence Scores

The anomaly score primitive  $\bar{s}_t$  can be converted to a Bayesian confidence estimate using the methods of [28]. With a Beta prior distribution, the expectation of the posterior distribution of an anomaly is

$$\rho_t = \frac{1 + \sum_{i=1}^{t-1} \mathbb{1}_{\{\bar{s}_i \leq \bar{s}_t\}}}{2 + (t-1)}. \quad (11)$$

Here,  $(t-1)$  is the number of graphs that have already been processed and  $\sum_{i=1}^{t-1} \mathbb{1}_{\{\bar{s}_i \leq \bar{s}_t\}}$  is the count of those with a primitive score less than or equal to  $\bar{s}_t$ .

From equation 6 in [28], anomaly confidence is given by

$$\pi_t = 1 - F_{\text{Bin}}[(t-1)(1-\gamma); (t-1), \rho_t] \quad (12)$$

where  $F_{\text{Bin}}[\cdot]$  is the Binomial cumulative distribution function and  $\gamma$  is a configurable “contamination” [28] factor.

Because the anomaly primitive scores are integers in  $[0, d]$ , a  $(d+1)$ -vector  $\mathbf{c}$  can be used to maintain the number of occurrences of each unique score value. Initialized as a sparse array of zeros, the  $(i+1)$ <sup>th</sup> element of  $\mathbf{c}$  is incremented by one every time a score  $\bar{s}_t = i$  occurs. Therefore, the number of scores less than or equal to  $\bar{s}_t$  at time  $t$  is  $\sum_{i=1}^{\bar{s}_t+1} \mathbf{c}_i$ . This structure provides efficient computation of  $\rho_t$ , which in turn

allows for efficient, bounded-memory estimation of anomaly confidences in a streaming environment.

The full procedure for maintaining the score counter  $\mathbf{c}$  and for computing the  $\bar{s}_t$  and  $\pi_t$  scores is given in algorithm 1.

#### E. Hyperparameters

When tuning hyperparameters for this approach, the relevant values to consider are the source and destination mapping probabilities  $p$  and  $q$  as well as the upper and lower anomaly thresholds  $\theta_L$  and  $\theta_U$ . The variance formula (equation 3 or 4) used to standardize encodings may also be considered in tuning or it can be set by assumption. The contamination factor  $\gamma$  is the (expected) proportion of anomalous snapshots present and is assumed through knowledge of the data.

The dimension of the hypervectors,  $d$ , affects the variance of the sketch statistic  $s_t$  and therefore the probability of error in anomaly detection. Large  $d$  is used to take advantage of concentration of measure. Consider  $s_t^2$ , referencing equation 8:

$$s_t^2 = \frac{1}{d} \sum_{i=1}^d \mathbf{z}_{t,i}^2$$

Each  $\mathbf{z}_{t,i}^2$  term is an independent random variable with mean 1. The variance of the sum of these variables is  $d \text{var}[\mathbf{z}_{t,i}^2]$ . Scaling by  $\frac{1}{d}$  has the result of scaling the variance by  $\frac{1}{d^2}$ , resulting in a variance  $\frac{\text{var}[\mathbf{z}_{t,i}^2]}{d}$ . The dimension  $d$  is set to a large value such as  $2^{10}$  or  $10^4$ , as is typical for hyperdimensional computing, and reduces the variance of  $s_t^2$ . This in turn increases the signal-to-noise ratio in the anomaly detector by reducing the variance of the noise.

The most important hyper-parameters to identify for this approach are  $\theta_L$  and  $\theta_U$ . A simple starting point is to flatten the standardized encodings of a subsample of  $\{\mathcal{G}_t\}$  into a single vector and examine the element-wise absolute values. For example, the flattened encodings of the first 10 snapshots in a sequence is  $\mathbf{z}^* = \text{vec}(\mathbf{z}_1, \dots, \mathbf{z}_{10})$  and the absolute values are defined  $\mathbf{z}_i^+ = |\mathbf{z}_i^*|$ . Then, the  $100\gamma^{\text{th}}$  and  $100(1-\gamma)^{\text{th}}$  percentiles of  $\mathbf{z}^+$  can be used for  $\theta_L$  and  $\theta_U$ .

### IV. EXPERIMENTS

To test our approach we conduct several experiments on both synthetic and real-world datasets. We first demonstrate the behavior of the encodings under different graph generating conditions. Next we demonstrate the effectiveness of our anomaly detection method on popular benchmarks.

We refer to the anomaly score primitives as HDSGA-p and the confidence scores as HDSGA-c. Results are compared against several existing techniques designed for streaming graphs. The approach from [20] is referred to as SpotLight. The approach from [11] is referred to as AnoGraph. The two approaches described in [9], MIDAS and MIDAS-R, are meant to be applied to individual edges. Thus, they are modified for comparison purposes by taking the average of all edge scores within snapshots.

#### A. Synthetic

1) *Data*: For synthetic experimentation, graph snapshots are generated as follows. For an arbitrary graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ , the sampling probability of for any dyad  $(u, v) \in \mathcal{V} \times \mathcal{V}$  is defined  $\alpha_{uv} \propto (\mathbf{d}\mathbf{d}^\top \odot \mathbf{A})_{u,v}$  where  $\mathbf{d}$  and  $\mathbf{A}$  are the degree vector and (binary) adjacency matrix of  $\mathcal{G}$ .

We consider 5 types of graphs from which snapshots are to be sampled: complete, caveman, star, wheel, and windmill. Sampling is done according to the probabilities described above. For simplicity, all edges have multiplicity (and therefore weight) of 1.

It should be noted that sampling from the complete graph corresponds to the case described in definition 1. This is also equivalent to the Erdős-Rényi model.

2) *Detection*: To begin, we generate 100 graph snapshots for each of the 5 graph types described in section IV-A1. For each snapshot, 100 unique edges are sampled according to the respective sampling probabilities. Each graph has 300 nodes. The windmill graph is constructed with 23 cliques of size 14. The caveman graph is constructed with 20 cliques of size 15.

The right side of figure 3 shows a 2-dimensional projection of the binary encodings  $\{\mathbf{b}_t\}$  computed for each snapshot. The projection is done with Principal Components Analysis (PCA). The visualization suggests that different types of graphs are mapped to different regions of the encoding space with some exceptions. Notice that there is overlap among the wheel and star snapshots as well as the caveman and complete snapshots. The wheel and star snapshots fail to be fully separated possibly because part of a wheel graph's decomposition is a star graph [29]. The caveman and complete snapshots fail to be separated likely because the caveman graph is itself a collection of multiple complete graphs; so the relationship in graph structures leads to similar sampling probabilities.

To simulate a streaming environment containing anomalous snapshots, we take 96 windmill graph snapshots for a baseline process and insert complete, star, caveman, and wheel graph snapshots at the 20<sup>th</sup>, 40<sup>th</sup>, 60<sup>th</sup>, and 80<sup>th</sup> indices. The left side of figure 3 shows the RMSEs, anomaly score primitives, and confidence scores computed from this stream. The RMSE trace suggests that the wheel and star snapshots are relatively further from expectation than the windmill snapshots and the caveman and complete snapshots are relatively closer (as hypothesized). These deviations are captured in the primitive score trace and made interpretable as anomalies in the confidence score trace.

The observation that snapshots sampled from similarly structured graphs are mapped to similar regions in the encoding space is a theoretical limitation of this approach. If the baseline process considered here instead consists of caveman snapshots, a complete snapshot would fail to be recognized as anomalous. In the next section, we demonstrate on several real-world datasets that this is not such a problem in the applied setting we examine (network traffic).

3) *Runtime*: Figure 4 shows the runtime results of each algorithm. The vertical axes represent execution time (averaged across 3 random seeds) on a set of 100 graph snapshots

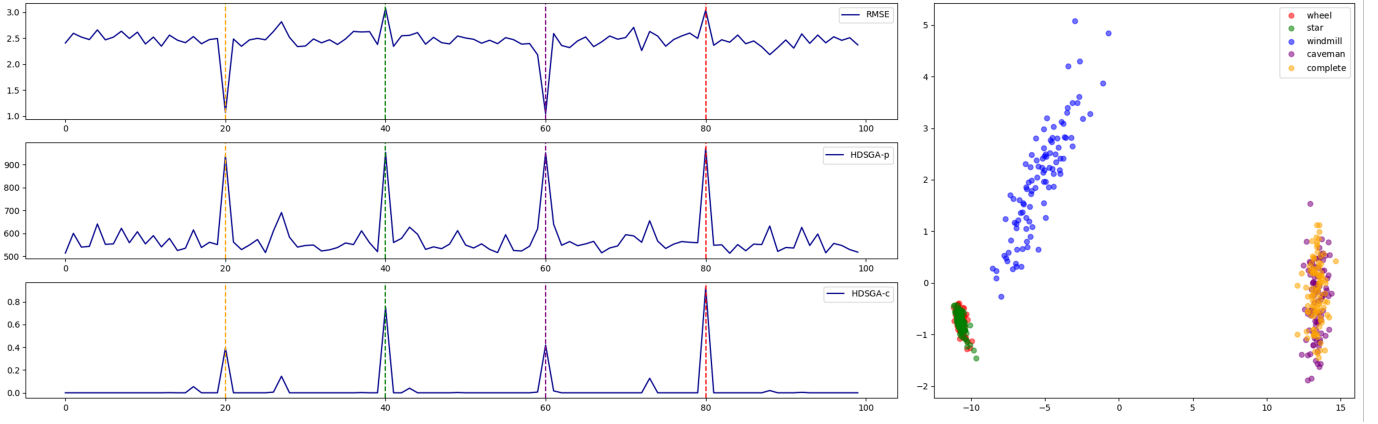


Fig. 3. Left: Anomaly scores (RMSE, HDSGA-p, and HDSGA-c) for a sequence of synthetic graph snapshots. Right: 2D projection (PCA) of binary encodings  $\{b_t\}$  for snapshots of different graph types.

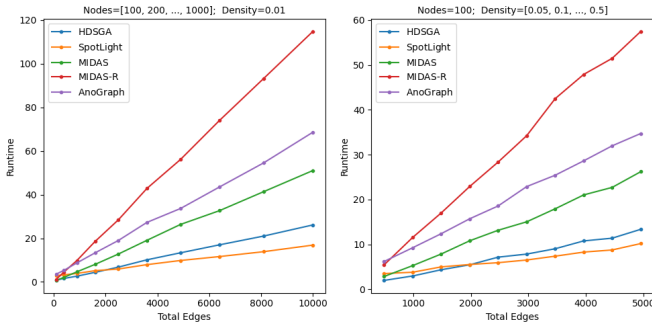


Fig. 4. Runtime results on synthetic data. Left: increasing graph order. Right: increasing graph density.

sampled from a complete graph. The horizontal axes represent the size of the graphs.

For the left-hand plot, density is kept fixed at 0.01 and the number of nodes (graph order) ranges from 100 to 1,000. For the right-hand plot, the number of nodes is kept fixed at 100 and graph density ranges from 0.05 to 0.5.

Each algorithm appears to scale roughly linearly with the number of edges. Notice that our approach is comparable to SpotLight in runtime and faster than the others.

## B. Real-World

1) *Data*: For real-world experimentation, we use the SIMARGL 2021 Network Intrusion Detection Dataset [30], the DARPA 1998 Intrusion Detection Evaluation Dataset [31], and the APPRAISE H2020 NetFlow Dataset [32].

The SIMARGL dataset has over 23M observations, around 9M of which are considered malicious. Malicious observations are categorized as “denial of service” (DoS), “malware”, and “port scanning”. For our experiments, edges are grouped into 1-minute windows for a total of 3,886 snapshots, with an average of roughly 1.1K unique nodes and 6.15K non-unique

edges per snapshot. A snapshot is labeled as anomalous if more than half of its edges are malicious; there are 556 anomalous snapshots in this dataset.

The DARPA dataset has over 4.5M observations, around 2.7M of which are malicious. Malicious observations are categorized as DoS, malware, and “network probe”. We group edges into 1-hour windows, affording 970 snapshots (88 of which are anomalous) with an average of roughly 150 unique nodes and 4.7K non-unique edges per snapshot.

The APPRAISE dataset has over 15M observations, around 4.3M of which are malicious. Malicious observations are categorized as DoS and “reconnaissance”. Edges are grouped into 6-hour windows for a total of 294 snapshots (14 of which are anomalous) with an average of roughly 550 unique nodes and 51.4K non-unique edges per snapshot.

Additionally, we use a simulated netflow dataset created with a generative masked transformer model called TabMT [33]. This dataset has 80K observations, around 21K of which are malicious. Edges are grouped into 10-second windows, resulting in 303 total snapshots (9 of which are anomalous) with approximately 130 unique nodes and 260 non-unique edges per snapshot.

2) *Detection*: For evaluation on real data, we use Area Under the Curve (AUC) of the Receiver Operator Characteristic (ROC) and Precision-Recall (PR) functions for evaluation metrics. These metrics are chosen because they do not require a prediction threshold.

Results on all four datasets are shown in the table I. The final column of the table is the average score over all four datasets. The best score for each dataset is highlighted in bold. Our anomaly primitives (HDSGA-p) and confidence estimates (HDSGA-c) achieve AUC-ROC of 95.73% and 93.65% on average. SpotLight achieves 85.55% on average. It should be noted that without aggregating labels and scores, MIDAS and MIDAS-R achieve average AUC-ROC of 95.04% and 96.98%.

Figure 5 visualizes the anomaly detection results on the

TABLE I  
ANOMALY DETECTION RESULTS REPORTED IN AREAS UNDER THE ROC AND PR CURVES.

|           | SIMARGL       |               | DARPA         |               | APPRAISE      |               | TabMT         |               | Average       |               |
|-----------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
|           | ROC           | PR            | ROC           | PR            | ROC           | PR            | ROC           | PR            | ROC           | PR            |
| HDSGA-p   | 0.9977        | 0.9859        | 0.8314        | 0.6220        | <b>0.9999</b> | <b>0.9999</b> | <b>0.9999</b> | <b>0.9999</b> | <b>0.9573</b> | <b>0.9020</b> |
| HDSGA-c   | 0.9952        | 0.9749        | 0.8129        | 0.6335        | 0.9463        | 0.9235        | 0.9917        | 0.8712        | 0.9365        | 0.8508        |
| SpotLight | 0.7188        | 0.3387        | 0.7934        | 0.5004        | 0.9117        | 0.7056        | 0.9981        | 0.9513        | 0.8555        | 0.6240        |
| AnoGraph  | <b>0.9996</b> | <b>0.9978</b> | 0.6784        | 0.4141        | 0.9997        | 0.9951        | <b>0.9999</b> | <b>0.9999</b> | 0.9195        | 0.8517        |
| MIDAS     | 0.9966        | 0.9608        | 0.7107        | 0.4252        | 0.9287        | 0.9319        | 0.9974        | 0.9498        | 0.9084        | 0.8169        |
| MIDAS-R   | 0.9976        | 0.9926        | <b>0.8395</b> | <b>0.6412</b> | 0.9287        | 0.9319        | <b>0.9999</b> | <b>0.9999</b> | 0.9414        | 0.8914        |

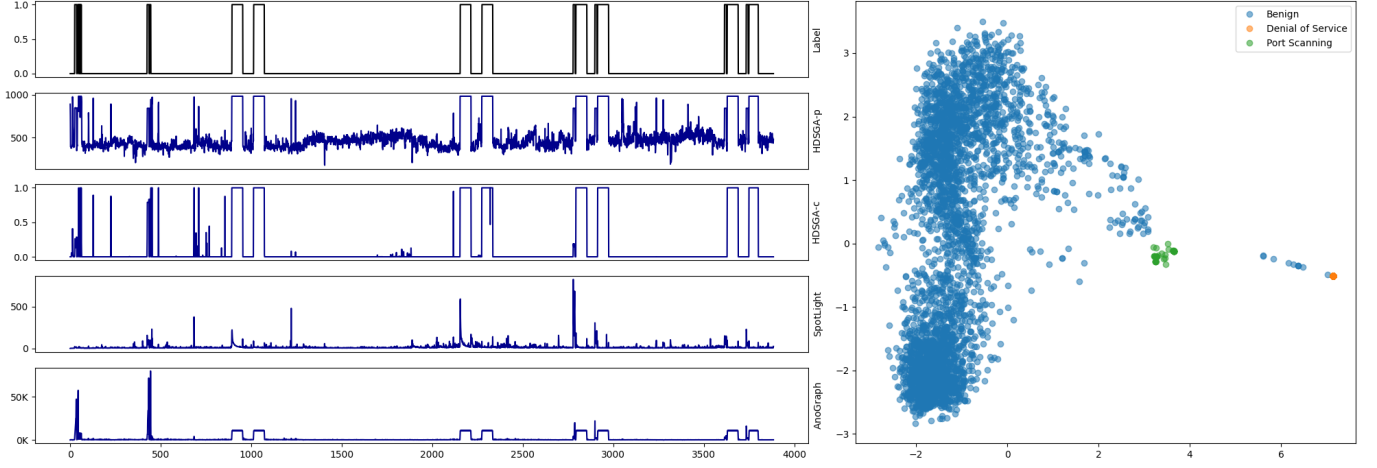


Fig. 5. Left: Anomaly detection results computed on the SIMARGL dataset; ground truth in the first row. Right: Projections of the binary encodings  $\{b_t\}$  for the SIMARGL dataset.

SIMARGL dataset. The left-hand side of the figure shows the time series of anomaly scores for each method with the ground truth labels in the first row. The left-hand side of the figure shows the 2-dimensional PCA projection of the binary encodings  $\{b_t\}$ . Note how the graphs corresponding to DoS (orange) and port scanning (green) attacks are mapped to individual clusters away from the benign (blue) graphs.

TABLE II  
RUNTIME RESULTS ON REAL-WORLD DATA.

|           | SIMARGL      | DARPA       | APPRAISE    | TabMT      |
|-----------|--------------|-------------|-------------|------------|
| HDSGA     | <b>374.9</b> | <b>24.4</b> | <b>20.7</b> | <b>3.9</b> |
| SpotLight | 543.0        | 74.3        | 35.6        | 12.0       |
| AnoGraph  | 1,456.5      | 295.6       | 952.9       | 12.6       |
| MIDAS     | 1,352.4      | 257.4       | 819.6       | 5.1        |
| MIDAS-R   | 2,392.3      | 472.8       | 1,641.1     | 8.6        |

3) *Runtime*: Table II shows the number of seconds it took for each algorithm to run on each dataset. On average, our approach runs  $1.88\times$  faster than the approach with the next shortest runtime. Taken together, these results demonstrate that our approach is just as accurate as those it is compared against, but executes in less time.

### C. Implementation

Experiments are written in Python and conducted on an AWS WorkSpaces virtual machine running Amazon Linux 2 OS with 7.7G of RAM, 11G of Swap, and a 2.5GHz Intel(R) Xeon(R) Platinum 8259CL CPU. We reproduce the implementation of SpotLight while leveraging the publicly available rrcf library<sup>2</sup>. We use the authors' python implementation<sup>3</sup> from [9] for both MIDAS and MIDAS-R. We use a third-party implementation<sup>4</sup> for AnoGraph.

For all experiments, the encoding dimension is  $d = 2^{10}$ . For the synthetic experiments, the algorithm is configured with parameters  $p = q = 0.2$ ,  $\theta_L = \theta_U = 2$ , and  $\gamma = 0.04$ . The variance used for standardized encodings is equation 4. For real-world experiments, parameters are chosen to maximize ROC while allowing reasonable runtime.

For experiments on the DARPA dataset, the algorithm is configured with parameters  $p = 0.3$ ,  $q = 0.2$ ,  $\theta_L = 0$ ,  $\theta_U = 3.5$ , and  $\gamma = 0.09$ . The variance used for standardized encodings is equation 3 (implying the assumption that the underlying graph is infinite). The comparison approaches are configured as they

<sup>2</sup>RRCF: <https://klabum.github.io/rrcf/>

<sup>3</sup>MIDAS: <https://github.com/ritesh99rakesh/pyMIDAS>

<sup>4</sup>AnoGraph: <https://github.com/VictorHoffmann1/AnoGraph-for-River-API>

appear in their original papers. This configuration is also used for the runtime experiments.

For experiments on the SIMARGL dataset, we change  $\theta_L = 1$ ,  $\theta_U = 4$ , and  $\gamma = 0.1$ . For SpotLight, the tree size is changed to 1,024. For AnoGraph, the edge threshold is set to 50.

For experiments on the APPRAISE dataset, our approach and SpotLight both use  $p = 0.2$  and  $q = 0.1$ . For SpotLight, the dimension and number of trees are changed to 100 and the tree size is changed to 512. For our approach, we change  $\theta_L = 0.1$ ,  $\theta_U = 3$ , and  $\gamma = 0.05$ . For AnoGraph, the edge threshold is again set to 50.

For experiments on the TabMT dataset, we use  $\theta_L = 0.5$  and  $\theta_U = 3$ . For SpotLight, tree size is 512. For MIDAS and MIDAS-R, we use 900 buckets for hashing.

## V. CONCLUSION

We have presented a non-parametric test for the randomness of the generating process of a graph snapshot. This test is adapted into a fast, low-memory, online algorithm for detecting anomalies in a graph stream. This is an early and promising exploration into the application of HDC for the graph setting. Configured properly, our approach is shown to be faster and just as accurate (or more so) than comparable approaches on real-world computer network datasets. As such, it is a valuable tool for a Zero Trust system.

## ACKNOWLEDGMENT

This research was conducted under Leidos' Office of Technology Zero Trust Mission Cloud Initiative, approval number 24-LEIDOS-0430-27761.

## REFERENCES

- [1] V. Stafford, "Zero trust architecture," *NIST special publication*, vol. 800, p. 207, 2020.
- [2] P. Kanerva, "Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors," *Cognitive Computation*, vol. 1, pp. 139–159, 2009. [Online]. Available: <https://api.semanticscholar.org/CorpusID:733980>
- [3] S. Ranshous, S. Shen, D. Koutra, S. Harenberg, C. Faloutsos, and N. F. Samatova, "Anomaly detection in dynamic networks: a survey," *Wiley Interdisciplinary Reviews: Computational Statistics*, vol. 7, no. 3, pp. 223–247, 2015.
- [4] L. Akoglu, H. Tong, and D. Koutra, "Graph-based anomaly detection and description: A survey," *CoRR*, vol. abs/1404.4679, 2014. [Online]. Available: <http://arxiv.org/abs/1404.4679>
- [5] X. Ma, J. Wu, S. Xue, J. Yang, C. Zhou, Q. Z. Sheng, H. Xiong, and L. Akoglu, "A comprehensive survey on graph anomaly detection with deep learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12 012–12 038, 2023.
- [6] P. Zhao, C. C. Aggarwal, and M. Wang, "gsketch: on query estimation in graph streams," *Proc. VLDB Endow.*, vol. 5, no. 3, p. 193–204, nov 2011. [Online]. Available: <https://doi.org/10.14778/2078331.2078335>
- [7] K. J. Ahn, S. Guha, and A. McGregor, "Graph sketches: sparsification, spanners, and subgraphs," in *Proceedings of the 31st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*, ser. PODS '12. New York, NY, USA: Association for Computing Machinery, 2012, p. 5–14. [Online]. Available: <https://doi.org/10.1145/2213556.2213560>
- [8] X. Gou, L. Zou, C. Zhao, and T. Yang, "Graph stream sketch: Summarizing graph streams with high speed and accuracy," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 6, pp. 5901–5914, 2023.
- [9] S. Bhatia, B. Hooi, M. Yoon, K. Shin, and C. Faloutsos, "Midas: Microcluster-based detector of anomalies in edge streams," 2020.
- [10] S. Ranshous, S. Harenberg, K. Sharma, and N. F. Samatova, "A scalable approach for outlier detection in edge streams using sketch-based approximations," in *Proceedings of the 2016 SIAM international conference on data mining*. SIAM, 2016, pp. 189–197.
- [11] S. Bhatia, M. Wadhwa, K. Kawaguchi, N. Shah, P. S. Yu, and B. Hooi, "Sketch-based anomaly detection in streaming graphs," 2023.
- [12] G. Cormode and S. Muthukrishnan, "An improved data stream summary: The count-min sketch and its applications," *J. Algorithms*, vol. 55, no. 1, p. 58–75, apr 2005. [Online]. Available: <https://doi.org/10.1016/j.jalgor.2003.12.001>
- [13] H. Yan, Q. Zhang, D. Mao, Z. Lu, D. Guo, and S. Chen, "Anomaly detection of network streams via dense subgraph discovery," in *2021 International Conference on Computer Communications and Networks (ICCCN)*, 2021, pp. 1–9.
- [14] D. Eswaran and C. Faloutsos, "Sedanspot: Detecting anomalies in edge streams," in *2018 IEEE International Conference on Data Mining (ICDM)*, 2018, pp. 953–958.
- [15] Y.-Y. Chang, P. Li, R. Sasic, M. H. Afifi, M. Schweighauser, and J. Leskovec, "F-fade: Frequency factorization for anomaly detection in edge streams," 2021.
- [16] E. Manzoor, S. M. Milajerdi, and L. Akoglu, "Fast memory-efficient anomaly detection in streaming heterogeneous graphs," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 1035–1044. [Online]. Available: <https://doi.org/10.1145/2939672.2939783>
- [17] P. Indyk and R. Motwani, "Approximate nearest neighbors: towards removing the curse of dimensionality," in *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, ser. STOC '98. New York, NY, USA: Association for Computing Machinery, 1998, p. 604–613. [Online]. Available: <https://doi.org/10.1145/276698.276876>
- [18] S. Huang, S. Coulombe, Y. Hitti, R. Rabbany, and G. Rabusseau, "Laplacian change point detection for single and multi-view dynamic graphs," 2023.
- [19] G. Chen, J. Arroyo, A. Athreya, J. Cape, J. T. Vogelstein, Y. Park, C. White, J. Larson, W. Yang, and C. E. Priebe, "Multiple network embedding for anomaly detection in time series of graphs," 2023.
- [20] D. Eswaran, C. Faloutsos, S. Guha, and N. Mishra, "Spotlight: Detecting anomalies in streaming graphs," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 1378–1386. [Online]. Available: <https://doi.org/10.1145/3219819.3220040>
- [21] S. Guha, N. Mishra, G. Roy, and O. Schrijvers, "Robust random cut forest based anomaly detection on streams," in *Proceedings of The 33rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. F. Balcan and K. Q. Weinberger, Eds., vol. 48. New York, New York, USA: PMLR, 20–22 Jun 2016, pp. 2712–2721. [Online]. Available: <https://proceedings.mlr.press/v48/guha16.html>
- [22] K. Schlegel, P. Neubert, and P. Protzel, "A comparison of vector symbolic architectures," *Artificial Intelligence Review*, vol. 55, 08 2022.
- [23] P. Poduval, H. Alimohamadi, A. Zakeri, F. Imani, M. H. Najafi, T. Givargis, and M. Imani, "Graphhd: Graph-based hyperdimensional memorization for brain-like cognitive learning," *Frontiers in Neuroscience*, vol. 16, 2022. [Online]. Available: <https://par.nsf.gov/biblio/10338293>
- [24] I. Nunes, M. Heddes, T. Givargis, A. Nicolau, and A. Veidenbaum, "Graphhd: Efficient graph classification using hyperdimensional computing," 2022.
- [25] M. Braei and S. Wagner, "Anomaly detection in univariate time-series: A survey on the state-of-the-art," 2020.
- [26] S. Schmidl, P. Wenig, and T. Papenbrock, "Anomaly detection in time series: A comprehensive evaluation," *Proc. VLDB Endow.*, vol. 15, pp. 1779–1797, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:250331153>
- [27] D. J. Schuurmann, "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability," *Journal of Pharmacokinetics and Biopharmaceutics*, vol. 15, pp. 657–680, 1987. [Online]. Available: <https://api.semanticscholar.org/CorpusID:206788664>

- [28] L. Perini, V. Vercruyssen, and J. Davis, "Quantifying the confidence of anomaly detectors in their example-wise predictions," in *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2020, Ghent, Belgium, September 14–18, 2020, Proceedings, Part III*. Berlin, Heidelberg: Springer-Verlag, 2020, p. 227–243. [Online]. Available: [https://doi.org/10.1007/978-3-030-67664-3\\_14](https://doi.org/10.1007/978-3-030-67664-3_14)
- [29] M. Subbulakshmi and I. Valliammal, "Decomposition of wheel graphs into stars, cycles and paths," *Malaya Journal of Matematik*, vol. 9, no. 01, p. 456–460, Jan. 2021. [Online]. Available: <https://www.malayajournal.org/index.php/mjm/article/view/1057>
- [30] M.-E. Mihailescu, D. Mihai, M. Carabas, M. Komisarek, M. Pawlicki, W. Hołubowicz, and R. Kozik, "The proposition and evaluation of the roedunet-simargl2021 network intrusion detection dataset," *Sensors*, vol. 21, no. 13, 2021. [Online]. Available: <https://www.mdpi.com/1424-8220/21/13/4319>
- [31] R. Lippmann, R. K. Cunningham, D. J. Fried, I. Graf, K. R. Kendall, S. E. Webster, and M. A. Zissman, "Results of the darpa 1998 offline intrusion detection evaluation," in *International Symposium on Recent Advances in Intrusion Detection*, 1999. [Online]. Available: <https://api.semanticscholar.org/CorpusID:32721274>
- [32] M. Komisarek, M. Pawlicki, T. Simic, D. Kavcnik, R. Kozik, and M. Choraś, "Modern netflow network dataset with labeled attacks and detection methods," in *Proceedings of the 18th International Conference on Availability, Reliability and Security*, ser. ARES '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3600160.3605094>
- [33] M. Gulati and P. Roysdon, "Tabmt: Generating tabular data with masked transformers," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

## APPENDIX A PROOFS

Here we examine properties of the  $i^{\text{th}}$  element of the encoding  $\mathbf{x}^\circ$  from definition 1 in section III-B. For simplicity, several notation adjustments are made.

Given the set of  $m$  edges  $\mathcal{E}^\circ$  and corresponding edge weights  $\mathcal{W}^\circ$ , let  $\mathcal{H}^\circ = \{\mathbf{H}_{uv,i} : (u,v,w) \in \mathcal{E}^\circ\}$  be a set of (scalar)  $\text{Bern}(\pi)$  variables, where  $\pi = pq$  is fixed across  $m$ . Note that all  $W_j \in \mathcal{W}^\circ$  and  $H_j \in \mathcal{H}^\circ$  are independent. We assume that  $W_j \geq 0$  for all  $j$ .

Furthermore, let  $X_j = W_j H_j$ . The variable we wish to examine is  $X^\circ = \sum_{j=1}^m X_j$ , which is equivalent to  $\mathbf{x}_{\cdot,i}^\circ$  from section III-B.

### A. Proposition 1

*Proof.* This proof aims to verify the expectation and variance of the (non-standardized) encoding of a *infinite* graph.

Recall that the conditional distribution (with respect to the edge weights) of the encoding of a snapshot from an infinite graph is a sum of weighted, independent Bernoulli variables.

For convenience, we let  $X_j|W_j=w_j$  be equivalently written  $X_j|w_j$ , where  $w_j$  is a realization of the variable  $W_j$ . So the conditional expectation of the encoding is

$$\mathbb{E}[X^\circ|\mathcal{W}^\circ] = \sum_{j=1}^m \mathbb{E}[X_j|w_j] \quad (13)$$

$$= \sum_{j=1}^m w_j \mathbb{E}[H_j] \quad (14)$$

$$= \pi \sum_{j=1}^m w_j \quad (15)$$

and the variance is

$$\text{var}[X^\circ|\mathcal{W}^\circ] = \sum_{j=1}^m \text{var}[X_j|w_j] \quad (16)$$

$$= \sum_{j=1}^m w_j^2 \text{var}[H_j] \quad (17)$$

$$= \pi(1-\pi) \sum_{j=1}^m w_j^2. \quad (18)$$

Note that when all weights are unity, this is equivalent to the first two central moments of the  $\text{Bin}(m, \pi)$  distribution.  $\square$

### B. Proposition 2

*Proof.* In this proof we aim to verify the expectation and variance of the (non-standardized) encoding of a *finite* graph.

Recall that the conditional distribution of the encoding of a snapshot from a finite graph is a sum of weighted, non-independent Bernoulli variables. Because the set of  $M$  edges from which the snapshot is sampled is finite, the set  $\mathcal{H}^\circ$  corresponding to the edge set is assumed to be a sample taken without replacement so that  $\text{cov}[H_j, H_s] = \frac{-\text{var}[H_j]}{M-1} = \frac{-\pi(1-\pi)}{M-1}$  for  $H_j, H_s \in \mathcal{H}^\circ$ .

Therefore, the conditional covariance of elements in the encoding summation is

$$\text{cov}[X_j, X_s|w_j, w_s] = \text{cov}[w_j H_j, w_s H_s|w_j, w_s] \\ = w_j w_s \text{cov}[H_j, H_s] \quad (19)$$

$$= w_j w_s \frac{-\pi(1-\pi)}{M-1} \quad (20)$$

and the conditional variance of the encoding is

$$\widetilde{\text{var}}[X^\circ|\mathcal{W}^\circ] = \sum_{j=1}^m \text{var}[X_j|w_j] + \sum_{j \neq s} \text{cov}[X_j, X_s|w_j, w_s] \\ = \pi(1-\pi) \sum_{j=1}^m w_j^2 + \frac{-\pi(1-\pi)}{M-1} \sum_{j \neq s} w_j w_s \quad (21)$$

$$= \pi(1-\pi) \frac{\lambda}{M-1} \quad (22)$$

$$\lambda = M \sum_{j=1}^m w_j^2 - \sum_{j=1}^m \sum_{s=1}^m w_j w_s. \quad (23)$$

The conditional expectation, being a linear operation, is unaffected by the dependence among elements in the summation; it is therefore the same as the infinite case. Note that when all weights are unity, this is equivalent to the second central moment of the  $\text{Hypergeom}(M, M\pi, m)$  distribution.  $\square$

### C. Proposition 3

*Proof.* We aim to demonstrate that each element of the standardized encoding of the infinite graph converges in distribution to a standard normal via the Lyapunov Central Limit Theorem. Each element of the encoding is a sum of independent but not identically distributed random variables:  $\sum_{j=1}^m X_j$  where  $X_j = w_j$  with probability  $\pi$  and 0 otherwise. Initially, the weights  $w_j$  will be arbitrary.

The mean of  $X_j$  is  $w_j\pi$  and the variance is  $\sigma_j^2 = \pi(1 - \pi)w_j^2$ . Since the random variables are independent, the variance of the sum is the sum of the variances:  $\sigma^2 = \sum_{j=1}^m \sigma_j^2$ .

The Lyapunov condition requires, for some  $\delta > 0$ ,

$$\lim_{m \rightarrow \infty} \frac{\sum_j \mathbb{E}[|X_j - \pi w_j|^{2+\delta}]}{\sigma^{(2+\delta)}} \rightarrow 0 \quad (24)$$

We will demonstrate the Lyapunov condition for  $\delta = 2$ . First, analyzing the numerator:

$$\begin{aligned} \mathbb{E} \left[ \sum_j (X_j - \pi w_j)^4 \right] &= \sum_{j=1}^m w_j^4 (\pi - 4\pi^2 + 6\pi^3 - 3\pi^4) \\ &= D \sum_{j=1}^m w_j^4 \end{aligned} \quad (25)$$

$$= D \|\mathbf{w}\|_4^4 \quad (26)$$

where  $D$  is a constant. If we consider  $\mathbf{w}$  to be a vector of dimension  $m$  (and  $\mathbf{w} \succeq 0$  as each element is non-negative) the summation can be identified as the 4<sup>th</sup> power of an  $\ell_4$  vector norm.

Next, consider  $\sigma^4 = (\sigma^2)^2$ :

$$\sigma^4 = \pi^2(1 - \pi^2) \left( \sum_{j=1}^m w_j^2 \right)^2 \quad (27)$$

Again, identifying  $\mathbf{w}$  as an  $m$ -dimensional vector,

$$\sigma^4 = C \|\mathbf{w}\|_2^4 \quad (28)$$

for some constant  $C$ . The Lyapunov condition to verify reduces to:

$$\lim_{m \rightarrow \infty} \frac{\|\mathbf{w}\|_4^4}{\|\mathbf{w}\|_2^4} \equiv \lim_{m \rightarrow \infty} \frac{\|\mathbf{w}\|_4}{\|\mathbf{w}\|_2} \rightarrow 0 \quad (29)$$

where  $\mathbf{w}$  is an  $m$ -dimensional vector; the dimension of  $\mathbf{w}$  increases as  $m \rightarrow \infty$  in this series.

Next, we show that this condition holds for bounded vectors (or bounded series of weights). Assume  $\mathbf{w} \succeq 0$  and  $\forall j \, w_j < \Gamma$  for some finite constant  $\Gamma$ . In the next portion, the weights will be sampled from binomial distributions, so we actually assume  $\mathbf{w} \succeq 1$ , where 1 is the minimum weight, and  $\forall j \, W_j < \Gamma$ . Assuming a finite probability of non-zero entries, or  $\|\mathbf{w}\| = o(1)$  implying  $\|\mathbf{w}\|$  grows much faster than a constant, we can remove 0-valued entries without loss of generality.

$$\begin{aligned} \frac{\|\mathbf{w}\|_4}{\|\mathbf{w}\|_2} &\leq \frac{\Gamma \sqrt[4]{m}}{\|\mathbf{w}\|_2} \\ &\leq \frac{\Gamma \sqrt[4]{m}}{\sqrt{m}} \\ &\Rightarrow \lim_{m \rightarrow \infty} \frac{\|\mathbf{w}\|_4}{\|\mathbf{w}\|_2} = 0 \end{aligned}$$

Thus, any sequence of independent weighted Bernoulli random variables with bounded non-negative weights that has enough non-zero entries to grow faster in norm than a constant, will satisfy the Lyapunov condition.  $\square$